

AI WORKFLOW PATTERNS FOR PRODUCT TEAMS

=====

Introduction

Modern product teams are integrating large language models (LLMs) into workflows that were previously manual, expensive, or slow. This document outlines common patterns, their trade-offs, and implementation considerations.

Pattern 1 Single prompt (zero-shot)

The simplest pattern: one user input, one LLM call, one output.

Use cases:

- Text summarisation
- Translation
- Classification
- Simple Q&A with known context

Trade-offs:

Pro: low latency, simple to implement, easy to test.

Con: limited reasoning depth, brittle for complex or multi-step tasks.

Pattern 2 Chain-of-thought prompting

The model is instructed to reason step by step before producing its final answer. This improves accuracy on tasks requiring multi-step logic.

Implementation note:

Add "Think step by step" or a structured reasoning prefix to the prompt.
For structured outputs, separate the reasoning trace from the final answer using XML delimiters or JSON schema.

Pattern 3 Tool use / function calling

The model is given a set of available tools (functions with JSON schemas).

It decides which tool to call, with which arguments, and the system executes the tool and returns the result for the model to continue.

Use cases:

- Reading from external APIs
- Running calculations
- Writing to databases
- File operations

Safety consideration: validate all model-generated tool arguments before execution. Never trust the model as the sole gatekeeper.

Pattern 4 Retrieval-Augmented Generation (RAG)

Instead of relying on the model's training data, RAG retrieves relevant passages from a document store and injects them into the prompt as context.

Pipeline steps:

1. Document ingestion: chunk, embed, and store documents in a vector index.
2. Query: embed the user question, run similarity search.
3. Retrieval: fetch top-k passages by cosine similarity score.
4. Augmentation: inject passages into the prompt as grounding context.
5. Generation: model answers based on retrieved context.

Key metrics:

- Retrieval recall: are the relevant passages being found?
- Answer groundedness: does the answer only use retrieved context?
- Latency: embedding + retrieval adds overhead before the LLM call.

Pattern 5 Multi-agent workflows

Multiple specialised agents handle different parts of a task and pass results between each other. A supervisor or orchestrator coordinates flow.

Example pipeline (content production):

Researcher -> Writer -> Critic -> Revision -> Groundedness check

Frameworks:

LangGraph: graph-based stateful agent orchestration

CrewAI: role-based agent teams

Custom: task queue with function-calling LLMs at each node

Trade-offs:

Pro: each agent can be specialised and independently tested.

Con: higher latency, harder to debug, failure modes compound.

Pattern 6 Human-in-the-loop

AI generates candidate outputs; a human reviews, edits, and approves before the output is used or sent downstream.

This pattern is critical for:

- High-stakes decisions (legal, medical, financial)
- Brand-sensitive outputs (marketing, customer communication)
- Tasks where AI error rate is too high for full automation

UX requirements:

- Make AI-generated content clearly distinguishable from human content
- Provide editable outputs, not read-only results
- Show confidence signals and source attribution where possible
- Enable fast rejection with one click

Security and safety considerations

Prompt injection: user-controlled content may contain instructions that attempt to override the system prompt. Mitigate by:

- Separating trusted instructions from untrusted user content with clear delimiters (XML tags, role-based messages)
- Validating model outputs before acting on them
- Never executing model-generated code without sandboxing

Data privacy: never send PII to external APIs without consent and

appropriate data processing agreements.

Rate limiting: LLM APIs have token and request limits. Build retry logic, back-off strategies, and per-user quotas into your system.

Evaluation

AI workflows are hard to test with conventional unit tests. Evaluation approaches include:

- Golden set: curated input/output pairs reviewed by humans
- LLM-as-judge: a second model scores outputs against criteria
- User feedback: thumbs up/down, edit rate, task completion rate
- Regression suite: run on every model or prompt change

Summary

The right AI workflow pattern depends on task complexity, latency budget, accuracy requirements, and safety constraints. Start with the simplest pattern that works, measure its failure modes, and upgrade complexity only when the simpler approach is proven insufficient.